

TradePilot

Autonomous trade finance & settlement for SMEs — built on Arc and USDC.

TradePilot turns an SME invoice into a programmable, milestone-based USDC escrow. An AI agent reads the invoice, proposes a payment workflow, and — once a human approves — drives real settlement through Circle's Developer-Controlled Wallets and a custom escrow contract deployed on Arc. Every completed trade builds a verifiable SME Credit Passport.

TRACK

**SME Trade Finance &
Working Capital**

CIRCLE ACCOUNT

maksutovdesign@icloud.com

LIVE DEMO

web-psi-rose-15.vercel.app

TRACK

Best SME Trade Finance & Working Capital Workflow (Invoices, Escrow, Settlement)

CIRCLE PRODUCTS USED ON ARC

USDC

Circle Wallets (Developer-Controlled)

Gateway — not used

CCTP/Bridge Kit — not used

USDC is the settlement and escrow asset throughout; Circle Wallets provides the SME buyer's embedded wallet, created via the live Circle API with real entity-secret registration.

FUNCTIONAL MVP

Frontend and backend are both deployed and live, wired to a real PostgreSQL database, a real Circle developer-controlled wallet, and a smart contract deployed on Arc Testnet — not a mock or local-only demo.

LIVE APP web-psi-rose-15.vercel.app

API api-sigma-hazel-15.vercel.app

GITHUB github.com/maksutovdesign/tradepilot

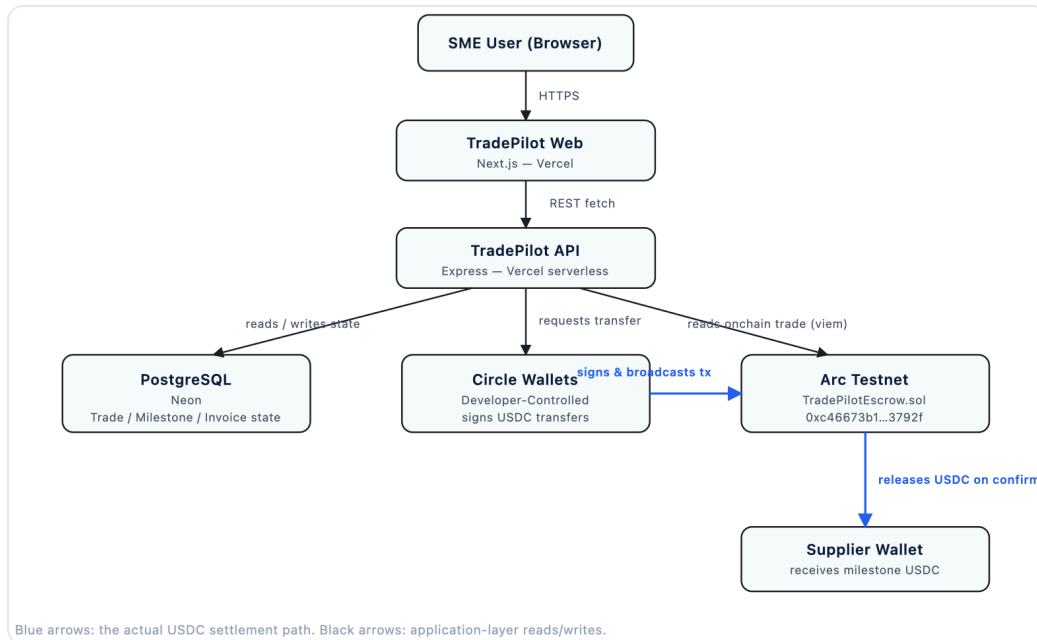
ESCROW CONTRACT (ARC TESTNET) `0xc46673b16c94d2898c59aeaa0fd588f2af13792f`

A full **create** → **fund** → **confirm** → **release** cycle has already been executed against this contract live on Arc Testnet:

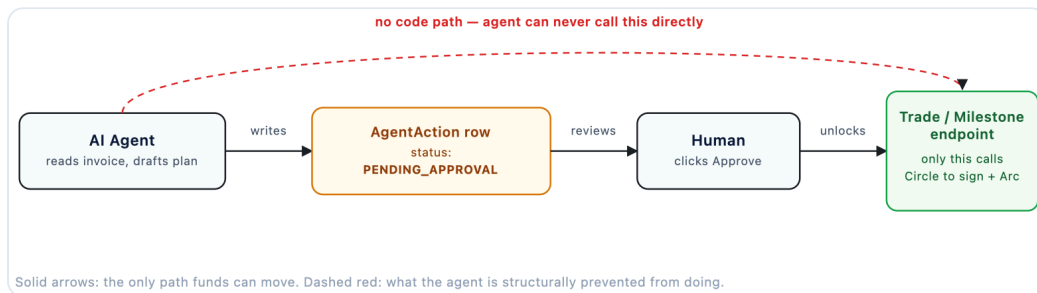
STEP	TX HASH
Create Trade (5 USDC, 2 milestones)	0xd695838e...5310e16535
Approve USDC	0x014f4fe0...49403576
Fund Trade	0xf0d691f8...6a63eee4c8701
Confirm + Release Shipment (2 USDC)	0x802358f3...144231a723
Confirm + Release Delivery (3 USDC) → TradeCompleted	0x13b59091...930af1a26

Full transaction links are in `docs/demo.md` in the repository and viewable live on testnet.arcsan.app.

ARCHITECTURE DIAGRAM



System architecture: browser → Next.js web → Express API → Postgres / Circle Wallets / Arc escrow contract → supplier wallet.



Agent trust boundary: the AI agent can only write a `PENDING_APPROVAL` row — it has no code path to sign a transaction. Only a human-approved request reaches Circle and Arc.

VIDEO DEMO

Submitted separately via the “Product Demo Video Link” field. The video walks through the full vertical slice: AI agent reads an invoice and proposes a milestone escrow → approval creates a real trade → escrow funded on Arc → milestones confirmed and released in USDC → Credit Passport updates — all against the live application and real onchain transactions.

DOCUMENTATION

Full setup instructions, environment variables, and a walkthrough of every Circle integration point (entity secret registration, developer-controlled wallet creation, USDC transfers) are in the GitHub repository:

SETUP + DEMO LINKS	README.md
ARCHITECTURE	docs/architecture.md
SMART CONTRACT	docs/smart-contract.md
CIRCLE INTEGRATION	docs/circle-integration.md
AGENT DESIGN	docs/agent.md
DEMO SCRIPT	docs/demo.md

Why we chose these products for your use case

Trade finance for SMEs needs two things banks are bad at: fast, predictable settlement, and a UX where the buyer never has to think about private keys. Developer-controlled wallets let TradePilot hold and move USDC on the company's behalf while a policy layer (AI proposes, human approves) decides when a transfer is allowed to happen. Arc's USDC-denominated fees make milestone amounts predictable, which matters when a \$30,000 shipment milestone can't be eaten into by gas volatility.

What worked well during development

The developer-controlled wallet model maps directly onto our trust boundary: the AI agent proposes, a human approves, and only then does a wallet-level call happen — we didn't have to build our own key-custody layer to get that separation. USDC's fixed 6-decimal precision made milestone-split math (30/40/30) exact, with no rounding surprises across onchain and database state.

What could be improved

The entity-secret ciphertext step (fetch public key → RSA-encrypt → attach to every signing call) is correct but easy to get subtly wrong on a first integration. We also hit a snag where our Circle Console account arrived with an entity secret already auto-registered during onboarding, with no visible way to recover or rotate it without the original recovery file — a clearer first-run flow (or an explicit "save this file now" warning) would help.

Recommendations

A reference architecture specifically for "AI agent proposes, backend executes" flows — including where spending limits and policy checks should live relative to the entity secret and wallet APIs — would make the agentic-commerce pattern much faster to get right, since every team building on developer-controlled wallets seems to reinvent that trust boundary independently.